Neuronal excitability and parameter variability in the Hodgkin-Huxley model.

This repository contains Python code for global sensitivity analysis of the Hodgkin-Huxley model, including voltage-clamp simulations and spatially-extended cable models. The code generates figures and analyses described in the associated manuscript.

Overview

The project performs Sobol sensitivity analysis on:

1. **Sodium (Na^+) gating parameters** under voltage-clamp
2. **Potassium (K^+) gating parameters** under voltage-clamp
3. **Full Hodgkin-Huxley model** with current injection (point neuron)
4. **Spatially-extended cable model** with current injection (10 cm cable)

Requirements

Python Libraries

Install the required libraries using pip:

```
pip install numpy scipy matplotlib pandas jax diffrax SALib
```

Core dependencies:

- `numpy` - Numerical computations
- `scipy` - ODE integration and statistical functions
- `matplotlib` - Plotting and visualization
- `pandas` - Data manipulation (Figure 1 analyses)
- `jax` - High-performance numerical computing with JIT compilation
- `diffrax` - JAX-based differential equation solvers
- `SALib` - Sobol sensitivity analysis

Note: JAX installation may require platform-specific instructions. See JAX installation guide.

File Descriptions

Main Analysis Scripts

Figure 1 Notebooks (`Figure 1/analyze_*.ipynb`)

Purpose: Bootstrap analysis of gating parameters from experimental data

- `analyze_ah.ipynb`, `analyze_am.ipynb`, `analyze_an.ipynb` - Sodium channel parameters (α and β functions)
- `analyze_bh.ipynb`, `analyze_bm.ipynb`, `analyze_bn.ipynb` - Potassium channel parameters (α and β functions)

Input Data: CSV files (`ah.csv`, `am.csv`, `an.csv`, `bh.csv`, `bm.csv`, `bn.csv`)

How to Run:

1. Navigate to the `Figure 1/` directory
2. Open any notebook in Jupyter
3. Run all cells sequentially

Key Output: Bootstrap parameter estimates (mean ± standard deviation)

`Figure2.ipynb` - Potassium Conductance Sensitivity

Purpose: Voltage-clamp simulation of K^+ conductance with Sobol analysis

Editable Parameters:

- **Voltage protocol** (lines 82-83): `python`
  ```
  t_steps = [0.0, 5.0, 30.0]      # Time points (ms)
  v_steps = [-80.0, -10.0, -80.0] # Voltage levels (mV)
  ```
  Format: Hold at v_steps[0] until t_steps[1], step to v_steps[1], return to v_steps[2] at t_steps[2]
- **Simulation parameters** (line 32): `python`
  ```
  tmax=50.0  # Total simulation time (ms)
  dt=0.1     # Time step (ms)
  ```

- **Sample size** (line 116): `python`
  ```
  N = 8192  # Number of Saltelli samples (creates ~360k parameter sets)
  ```

How to Run: `bash`
`jupyter notebook Figure2.ipynb`
Run all cells. Execution time: ~5-10 minutes.

Outputs:

- `all_g_k.csv` - Conductance traces
- `S1_time_k.csv`, `ST_time_k.csv` - Sobol sensitivity indices
- `mean_activation_gate_n.csv` - Average conductance
- Plots: K^+ conductance traces and time-dependent sensitivity

---

`Figure3.ipynb` - Sodium Conductance Sensitivity

Purpose: Voltage-clamp simulation of Na^+ conductance with Sobol analysis

Editable Parameters:

- **Voltage protocol** (lines 112-113): `python`
  ```
  t_steps = [0.0, 5.0, 30.0]      # Time points (ms)
  v_steps = [-80.0, -10.0, -80.0] # Voltage levels (mV)
  ```
- **Simulation parameters** (line 32): `python`
  ```
  tmax=50.0  # Total simulation time (ms)
  dt=0.05    # Time step (ms)
  ```
- **Sample size** (line 140): `python`
  ```
  N = 4096  # Number of Saltelli samples
  ```

How to Run: `bash`
`jupyter notebook Figure3.ipynb`
Run all cells. Execution time: ~10-20 minutes.

Outputs:

- `all_g_na.csv` - Conductance traces
- `S1_time_na.csv`, `ST_time_na.csv` - Sobol sensitivity indices
- `mean_g_na.csv` - Average conductance
- Plots: Na^+ conductance traces and time-dependent sensitivity

---

`Figures 4-7/` - *Spatially-Extended Cable Model*

Purpose: Monte Carlo simulation of 10 cm axon cable with parameter variation

Editable Parameters:

- **Current injection** (in `parameters.py` or command line): `python`
  `I_AMPLITUDE = 2.0  # Injected current amplitude (μA/cm²)`
  Recommended range: 1-60 μA/cm²
- **Cable geometry** (in `config.py`): `python`
  `L_AXON = 10.0      # Axon length (cm)`
  `A_RADIUS = 0.025   # Axon radius (cm) [0.5 mm diameter]`
  `N_SEGMENTS = 80    # Number of spatial segments`
- **Simulation parameters** (in `config.py`): `python`
  `tmax=100.0  # Total simulation time (ms)`
  `dt=0.005    # Time step (adaptive, calculated automatically)`
- **Sample size** (command line or default): `bash`
  `python main.py --n-samples 300000`

How to Run: `bash`
`python Figures\ 4-7/main.py --n-samples 300000`

Execution time: ~2-4 hours for N=300,000 (uses JAX acceleration with 11 CPU cores)

Outputs:

- Histograms: Resting potential, firing frequency, stable potential, AP amplitudes
- Pie chart: Trace categorization (no AP, single AP, multiple AP, spontaneous)
- Representative traces for each category
- Parameter distributions by firing category
- Propagation analysis: AP amplitudes and conduction velocity
- CSV files: Categorization results, parameter values, average membrane potentials

---

`Figure 8/` - *Point Neuron Sobol Sensitivity*

Purpose: Time-dependent Sobol analysis of point HH model with current injection

Editable Parameters:

- **Current injection** (in `config.py`): `python I_amplitude=30.0  # Injected current (μA/cm²)` Recommended range: 1-60 μA/cm²
- **Simulation parameters** (in `config.py`): `python tmax=100.0  # Total simulation time (ms) dt=0.005    # Time step (ms)`
- **Sample size** (in `config.py` or default): `python N = 8192  # Sobol samples (creates ~373k parameter sets with Saltelli sampling)`

How to Run: `bash python Figure\ 8/pipeline.py`
Execution time: ~1-2 hours (JAX-accelerated)
Outputs:

- Voltage traces with stimulus period overlay
- Time-dependent first-order (S_1) and total-order (S_t) Sobol indices
- CSV files: Mean/std voltage, Sobol indices, sample traces
- Plots showing parameter sensitivity during AP generation

Data Files

- **`params_boot.csv`** - Bootstrap parameter estimates (mean, std) for all 22 HH parameters
- Used by `Figures 4-7/` and `Figure 8/` to generate parameter distributions
- Format: `mean, std # parameter_name`

Folder Structure

The codebase is organized into modular folders for different analyses:

- **`Figure 1/`** - Bootstrap analysis of gating parameters from experimental data
- Notebooks: `analyze_ah.ipynb`, `analyze_am.ipynb`, etc.
- Data: CSV files for each parameter
- **`Figure 8/`** - Point neuron Sobol sensitivity analysis
- Modular scripts: `pipeline.py` (main), `sim_core.py`, `analysis.py`, etc.

- Configuration: `config.py`, `parameters.py`
- **`Figures 4-7/`** - Spatially-extended cable model Monte Carlo analysis
- Modular scripts: `main.py` (main), `core_simulation.py`, `core_analysis.py`, etc.
- Configuration: `config.py`, `parameters.py`
- **Root level** - Voltage-clamp sensitivity notebooks (`Figure2.ipynb`, `Figure3.ipynb`) and shared data (`params_boot.csv`)

Key Concepts

Parameter Modification

For Current Injection Studies (Figures 4-8):

- Modify `I_amplitude` or `I_AMPLITUDE` variable
- Units: μA/cm² (current density)
- Physiological range: 1-60 μA/cm²
- Lower values: Subthreshold responses
- Higher values: Multiple action potentials

For Voltage-Clamp Studies (Figures 2-3):

- Modify `t_steps` and `v_steps` arrays
- `t_steps`: Time points for voltage transitions (ms)
- `v_steps`: Holding/step voltage levels (mV)
- Example protocols:
- Standard activation: [-80, -10, -80] mV
- Inactivation: [-80, -40, -10, -80] mV (add extra step)

Computational Performance

JAX Optimization:

- All `.py` scripts use JAX for 10-50× speedup vs. pure NumPy/SciPy
- JIT compilation on first run (slower), subsequent runs are fast
- Automatic multi-core parallelization (configurable via `XLA_FLAGS`)

Memory Management:

- Large simulations use chunked processing to avoid memory overflow
- Chunk files saved to `temp_*_results/` directories

- Automatic cleanup after analysis (controlled by `KEEP_SIMULATION_DATA` flag)

Recommended Computing Resources:

- RAM: ≥16 GB for full-scale simulations (N=300,000)
- CPU: Multi-core processor (8+ cores recommended)
- Disk space: ~2-5 GB temporary storage during execution

Typical Workflow

Quick Test (Voltage-Clamp)

1. Open `Figure2.ipynb` or `Figure3.ipynb`
2. Reduce `N` to 1024 for quick testing
3. Run all cells (~2 minutes)

Full Analysis (Cable Model)

1. Ensure `params_boot.csv` exists
2. Edit `I_AMPLITUDE` in `Figures 4-7/parameters.py` or via command line if desired
3. Run: `python Figures\ 4-7/main.py --n-samples 300000`
4. Wait for completion (~2-4 hours)
5. Review generated figures (PDF format)

Sensitivity Analysis (Point Neuron)

1. Edit `I_amplitude` in `Figure 8/config.py` if desired
2. Run: `python Figure\ 8/pipeline.py`
3. Wait for completion (~1-2 hours)
4. Analyze Sobol indices showing parameter importance over time

Troubleshooting

JAX Installation Issues:

- On macOS: `pip install jax[cpu]`
- On Linux/Windows: Follow platform-specific instructions at https://github.com/google/jax

Memory Errors:

- Reduce `N` (sample size)
- Reduce `chunk_size` in incremental simulation functions
- Close other applications to free RAM

Slow Execution:

- JAX requires compilation on first run (normal)
- Ensure multi-core support is enabled
- Consider reducing `N` for testing

Missing Output Files:

- Check console for error messages
- Verify all required libraries are installed
- Ensure sufficient disk space for temporary files

Citation

If you use this code, please cite the associated manuscript

Contact

For questions or issues, please refer to the manuscript or contact the authors.